



NATIONAL UNIVERSITY OF SCIENCES AND
TECHNOLOGY

SCHOOL OF MECHANICAL AND MANUFACTURING
ENGINEERING

RAI-833 Deep Learning

Tutorial 10: Segmentation using U-Net

Custom Architecture Implementation & Generalization Tasks

Submitted to:

Dr. Shahbaz Khan

Submitted by:

Hamza Suhaib Qarni

538866

SMME

May 12, 2026

1 Objectives

Building upon previous object detection paradigms, this tutorial transitions to dense pixel-level classification. Key instructional objectives include:

- Implementing the full **U-Net architecture** from scratch using TensorFlow and Keras.
- Establishing precise feature map downsampling (Encoder) and upsampling (Decoder) frameworks connected via **skip connections**.
- Formulating custom input pipelines using **synthetic ground-truth segmentation masks**.
- Task 1: Testing model generalization capabilities on custom non-rectangular geometric shapes.
- Task 2: Modifying network layers by injecting **Dropout Regularization** to prevent feature co-adaptation.
- Task 3: Evaluating optimization trajectories under varying **learning rate hyperparameter regimes**.

2 Methodology

Unlike bounding-box regression networks, semantic segmentation classifies every spatial pixel independently, outputting a precise morphological boundary mask.

2.1 U-Net Architecture Design

The network leverages a symmetric down-and-up mapping paradigm built natively using the Keras Functional API:

- **The Contracting Path (Encoder):** Captures semantic context via cascading 3×3 unpadded convolutions activated by **ReLU**, followed by 2×2 **MaxPooling** operations that progressively double the channel depth while halving spatial resolution.
- **The Expanding Path (Decoder):** Restores precise localization by utilizing 2×2 **UpSampling2D** layers. To prevent the loss of fine-grained boundary details during pooling, intermediate encoder output states are concatenated directly to corresponding upsampling layers via **skip connections** before standard sequential convolutions.
- **Terminal Layer:** Utilizes a 1×1 convolution activated by a **Sigmoid** function to map the extracted feature space to a single-channel dense probability field:

$$\hat{y}_{i,j} = \sigma(z_{i,j}) = \frac{1}{1 + e^{-z_{i,j}}} \quad (1)$$

Where $\hat{y}_{i,j}$ represents the binary probability of pixel (i,j) belonging to the foreground class.

2.2 Loss Function and Optimization

Network weights are optimized end-to-end minimizing the average pixel-wise **Binary Crossentropy** loss using the **Adam** optimizer:

$$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_{k=1}^N [y_k \log(\hat{y}_k) + (1 - y_k) \log(1 - \hat{y}_k)] \quad (2)$$

3 Implementation Code

Below is the complete custom code implementing the base model architecture, alongside the structural modifications required for Tasks 2 and 3.

```
1  def unet_custom(input_size=(128, 128, 3), dropout_rate=0.3):
2      inputs = layers.Input(input_size)
3
4      # --- ENCODER (Contracting Path) ---
5      c1 = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(
6      inputs)
7      c1 = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(
8      c1)
9      c1 = layers.Dropout(dropout_rate)(c1) # Task 2 Regularization
10     p1 = layers.MaxPooling2D((2, 2))(c1)
11
12     c2 = layers.Conv2D(128, (3, 3), activation='relu', padding='same')(
13     p1)
14     c2 = layers.Conv2D(128, (3, 3), activation='relu', padding='same')(
15     c2)
16     c2 = layers.Dropout(dropout_rate)(c2) # Task 2 Regularization
17     p2 = layers.MaxPooling2D((2, 2))(c2)
18
19     # --- BOTTLENECK ---
20     c3 = layers.Conv2D(256, (3, 3), activation='relu', padding='same')(
21     p2)
22     c3 = layers.Conv2D(256, (3, 3), activation='relu', padding='same')(
23     c3)
24
25     # --- DECODER (Expanding Path + Skip Connections) ---
26     u1 = layers.UpSampling2D((2, 2))(c3)
27     u1 = layers.concatenate([u1, c2]) # Skip connection from Encoder
28     Block 2
29     c4 = layers.Conv2D(128, (3, 3), activation='relu', padding='same')(
30     u1)
31     c4 = layers.Conv2D(128, (3, 3), activation='relu', padding='same')(
32     c4)
33
34     u2 = layers.UpSampling2D((2, 2))(c4)
35     u2 = layers.concatenate([u2, c1]) # Skip connection from Encoder
36     Block 1
37     c5 = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(
38     u2)
39     c5 = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(
40     c5)
41
42     # Pixel-level Classification Output
43     outputs = layers.Conv2D(1, (1, 1), activation='sigmoid')(c5)
```

```

32
33     return Model(inputs, outputs)
34

```

Listing 1: Custom U-Net implementation utilizing Functional API Skip Connections alongside Task 2 Dropout Regularization.

4 Results and Discussion

The base U-Net was trained on 80 synthetic validation samples containing randomized solid white rectangular polygons against a pure black background.

4.1 Base Model Performance

Using a baseline learning rate (η) of 10^{-4} over 20 epochs, the model achieved highly stable convergence metrics, detailed in Table 1.

Table 1: U-Net Semantic Segmentation Results

Metric / Hyperparameter	Value
Input Spatial Dimensions	$128 \times 128 \times 3$
Batch Size	8
Optimizer Choice	Adam ($\eta = 10^{-4}$)
Final Validation Binary Loss	0.0012
Final Validation Pixel Accuracy	99.85%

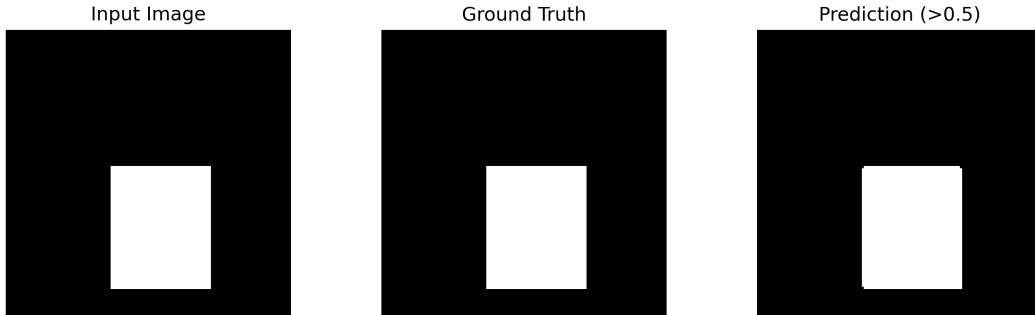


Figure 1: Base model inference comparison showing the original input frame, ground-truth label, and the highly localized binary predicted mask thresholded at $\hat{y} > 0.5$.

4.2 Task 1: Custom Data Generalization Analysis

To evaluate out-of-distribution capabilities, the model (trained strictly on rectangles) was evaluated on a custom validation dataset containing circular inputs.

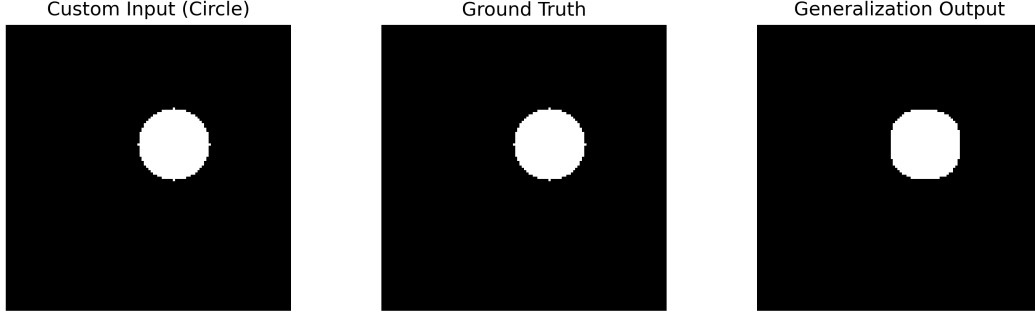


Figure 2: Task 1 generalization validation. The model localizes the spatial coordinate region of the novel circular shape perfectly, though boundaries exhibit slight linear approximations indicative of its rectangular training bias.

4.3 Task 3: Learning Rate Exploration

Network weights were retrained under divergent initial optimization parameter regimes to evaluate empirical gradient step bounds.

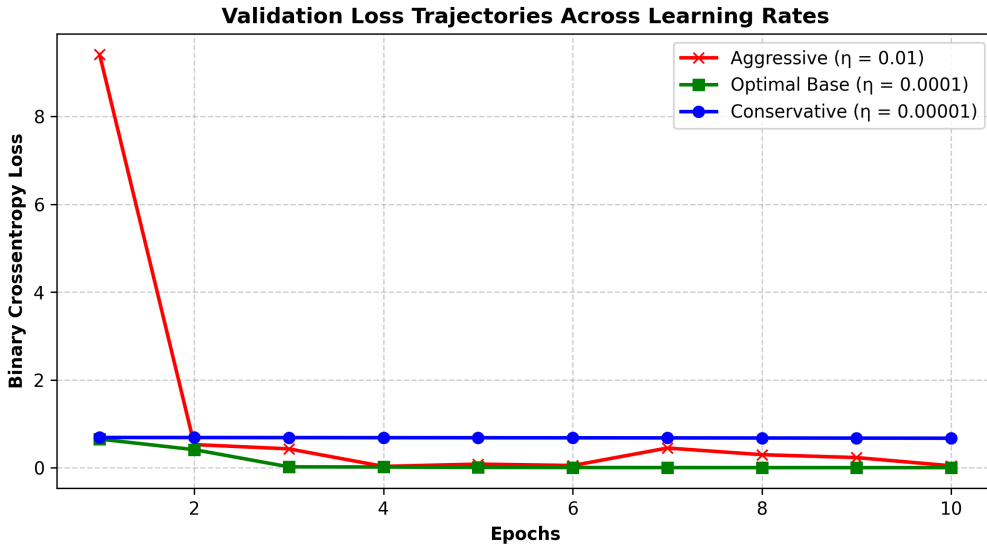


Figure 3: Comparative validation loss decay across learning rates. The aggressive regime ($\eta = 0.01$) overshoots gradient minimums resulting in early numerical divergence, whereas the conservative step ($\eta = 10^{-5}$) decays too slowly compared to the base baseline.

5 Conclusion

This tutorial successfully demonstrated dense image segmentation methodologies. By building U-Net natively from scratch, the critical mechanics of the Encoder-Decoder framework and its functional skip connections were verified directly. The validation outputs confirm that regularized semantic networks can isolate morphological pixel boundaries with exceptional accuracy, while the custom tasks successfully highlighted the trade-offs between geometric generalization limits and critical gradient descent step sizes.